Machine Learning — Lecture 3

Linear Regression, Optimization, and Regularization

FCIS — For Course Use Only

Dr. Fayçal Hamdi

Faculty of Computer and Information Systems

Lecture 3 at a Glance

FCIS — Course Use Only

Linear Regression, Optimization, and Regularization

Course Learning Outcomes

CLO3: Construct and analyze machine learning models by applying algorithmic principles and internal computations.

CLO4: Apply supervised and unsupervised learning techniques to solve practical problems and interpret their outcomes.

Lecture purpose

- Introduce the first full supervised-learning model.
- Connect prediction, residuals, loss, and optimization.
- Derive how gradient descent updates model parameters.
- Explain how regularization controls model flexibility.
- Evaluate regression models using appropriate metrics.

FCIS — For Course Use Only

💡 Main idea: linear regression is simple, but it contains the complete learning pattern: **model → prediction → error → loss → optimization → evaluation**.

Learning Objectives

FCIS — Course Use Only

By the end of Lecture 3, you will be able to:

1. Formulate regression as a supervised-learning problem.
2. Represent simple and multiple **linear regression** mathematically.
3. Compute predictions, residuals, and squared errors.
4. Explain Mean Squared Error as an objective function.
5. Describe how gradient descent reduces loss.
6. Perform one manual parameter update for a simple linear model.
7. Distinguish batch, stochastic, and mini-batch gradient descent.
8. Explain overfitting through model flexibility and polynomial features.
9. Explain Ridge and Lasso regularization conceptually.
10. Evaluate regression models using MAE, MSE, RMSE, and R^2, and justify a model choice on validation data.

Lecture Roadmap

FCIS — Course Use Only

From a line to a learning algorithm

Learning phase	Parts	Core question
1. Formulate the model	**I–II**	What are we predicting, and how does a linear model produce a prediction?
2. Learn the parameters	**III–IV**	How do we measure errors and reduce a loss?
3. Control complexity	**V–VI**	When do flexible models overfit, and how does regularization help?
4. Evaluate and apply	**VII–VIII**	How do we judge a regression model on unseen cases?

FCIS — For Course Use Only

💡 **Practice & consolidation:** calculate predictions, recognize overfitting and common mistakes, and explain defensible modeling decisions.

Part I

FCIS — For Students Use Only

Regression as Supervised Learning

Regression is a supervised-learning task that predicts a **numerical target** from one or more **input features**.

FCIS — For Course Use Only

What is a Regression Problem?

FCIS Internal Use Only

Predicting a numerical quantity

In **supervised learning**, we observe labeled examples (x_i, y_i) and learn to predict a target from the input features.

For regression, the prediction $\hat{y}_i$ is a **numerical value**.

Common applications

Problem	Input features	Target
Apartment rent	Area, district, rooms, age	Monthly rent
Delivery time	Distance, traffic, weather	Minutes
Exam performance	Study hours, attendance, GPA	Final score
Customer spending	Orders, visits, recency	Spending amount

FCIS — For Course Use Only

> **Regression** predicts a **numerical quantity**, whereas classification predicts a **category or class label**.

The Learning Task

FCIS — Academic Use Only

Approximate an unknown relationship

The true relationship between features and target is usually unknown:

$$\boxed{y = f(x) + \varepsilon}$$

where:

- $f(x)$: The underlying relationship between inputs and target.
- ε: Noise, measurement error, or unobserved influences.

A regression model learns an approximation:

$$\hat{y} = \hat{f}(x)$$

FCIS — For Course Use Only

What is the objective?

Learn a model that **generalizes to unseen observations**, rather than simply fitting the training examples.

> 💡 **Key idea.** A rent-prediction model should estimate prices for **new apartments**, not merely reproduce rents from its training dataset.

Dataset Representation

FCIS — Private Use Only

Feature matrix and target vector

A regression dataset contains n **observations** and p **input features**:

$$X = \begin{bmatrix} x_{11} & \cdots & x_{1p} \\ \vdots & \ddots & \vdots \\ x_{n1} & \cdots & x_{np} \end{bmatrix}, \qquad y = \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix}$$

Understanding the structure

Symbol	Meaning
$X \in \mathbb{R}^{n \times p}$	Feature matrix
$y \in \mathbb{R}^n$	Target vector
Row of X	One observation
Column of X	One input feature

FCIS — For Course Use Only

💡 **Key idea.** During training, the model learns the relationship $X \rightarrow y$. For new observations, it uses their features to produce predictions $\hat{y}$.

Example Dataset

FCIS Lecture Use Only

Predicting final exam scores

Suppose we have the following **illustrative student dataset**:

Student	Study hours	Attendance (%)	Previous GPA	Final score
A	3	60	2.1	55
B	6	75	2.8	68
C	9	88	3.2	81
D	12	95	3.6	90

FCIS For Course Use Only

Supervised-learning formulation

- **Features (X):** Study hours, attendance, previous GPA.
- **Target (y):** Final exam score.
- **Task:** Regression.
- **Prediction:** Estimated score for a new student.

Avoid target leakage. The final exam score is the value we want to predict—it must **not be included among the input features**.

Residuals

FCIS — Private Use Only

Measuring prediction errors

A **residual** is the difference between an observed target and its predicted value:

$$\boxed{e_i = y_i - \hat{y}_i}$$

How do we interpret residuals?

Residual	Interpretation
$e_i > 0$	Model **underpredicted**
$e_i < 0$	Model **overpredicted**
$e_i = 0$	Prediction is exact

FCIS — For Course Use Only

Example: If the actual exam score is **80** and the prediction is **74**, the residual is $80 - 74 = +6$.

Why do residuals matter?

Residuals help us **measure prediction errors**, construct loss functions, and diagnose model limitations.

> **Key caution.** Systematic patterns in residuals may indicate **missing features, nonlinear relationships, or an unsuitable model**.

Part II

FCIS — For Private Use Only

Simple Linear Regression

Simple linear regression models the relationship between **one input feature** and a **continuous target** using a straight line.

FCIS — For Course Use Only

Simple Linear Model

FCIS — Private Use Only

One feature, one target

Simple **linear regression** predicts a numerical value using:

$$\hat{y} = w_0 + w_1 x$$

where x is the **input feature** and $\hat{y}$ is the **predicted target**.

Interpreting the parameters

Parameter	Meaning
w_0 — **Intercept**	Predicted value when $x = 0$
w_1 — **Slope**	Change in the prediction for a one-unit increase in x

ECIS — For Course Use Only

Example: Predicting rent

If $\hat{y} = 1500 + 40x$, where x is apartment area in m²:

- **Intercept:** 1500 is the model's prediction at $x = 0$.
- **Slope:** Each additional m² increases predicted rent by **40 monetary units**.

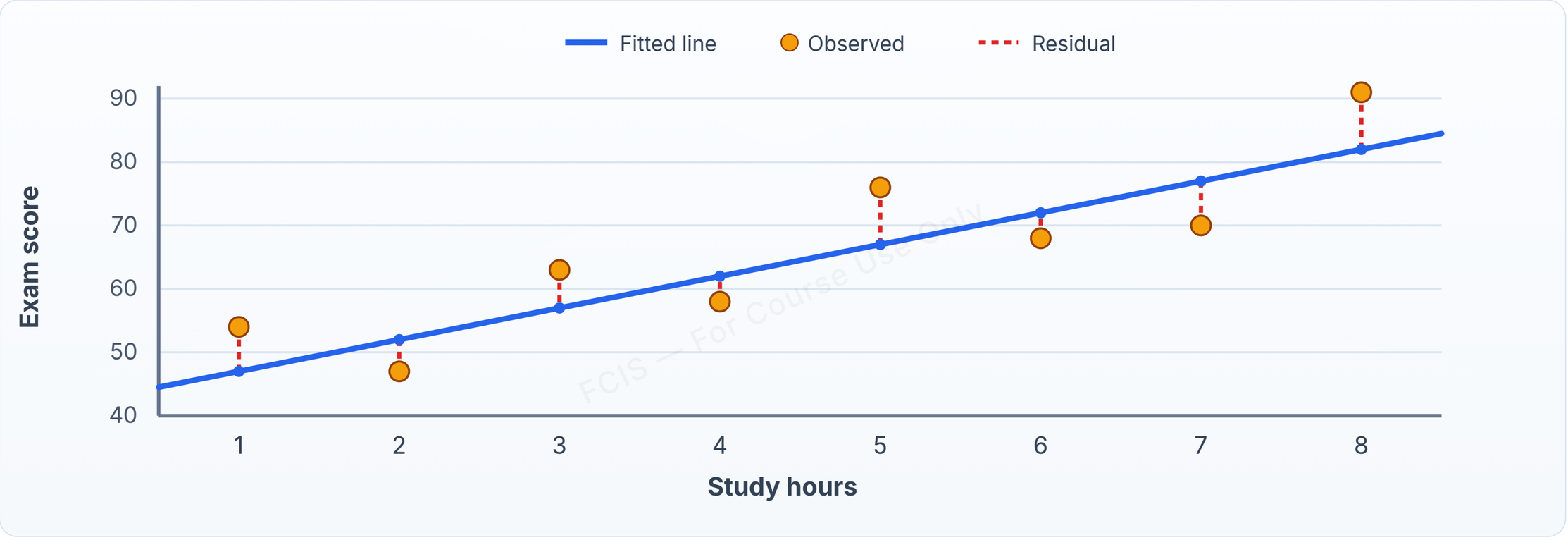
> 💡 **Key idea.** The intercept sets the line's starting level; the slope determines its **direction and steepness**. Interpret the intercept cautiously when $x = 0$ is not meaningful.

Visual Interpretation

FCIS — For Educational Use Only

A fitted line and its residuals

Each orange point is an **observed exam score**. The blue line represents the **predicted score**, and each red segment shows the vertical **residual**.



FCIS — For Course Use Only

💡 **Reading residuals:** A point **above** the line has a positive residual (**underprediction**); a point **below** has a negative residual (**overprediction**).

Worked Prediction Example

FCIS — Internal Use Only

Compute a prediction and residual

Suppose a regression model predicts exam score from **study hours**:

$$\hat{y} = 40 + 4x$$

For a student who studied **6 hours** and scored **68**:

Step 1: Predict the score

$$\hat{y} = 40 + 4(6) = \boxed{64}$$

FCIS — For Course Use Only

Step 2: Calculate the residual

$$e = y - \hat{y} = 68 - 64 = \boxed{+4}$$

Interpretation. The model **underpredicted** the student's score by **4 points**. The observed score lies above the prediction line.

Multiple Linear Regression

FCIS — For Course Use Only

Many features, one target

Multiple linear regression extends simple linear regression by using several input features:

$$\boxed{\hat{y} = w_0 + w_1 x_1 + \cdots + w_p x_p}$$

In vector notation:

$$\hat{y} = w^\top x + b$$

where w contains the **feature coefficients** and $b = w_0$ is the **intercept**.

Example: Predicting exam scores

$$\widehat{\text{score}} = 30 + 3(\text{hours}) + 0.2(\text{attendance}) + 5(\text{GPA})$$

Each coefficient describes the change in the **predicted score** associated with its feature, **holding the other features fixed**.

> 💡 **Key idea.** Multiple regression combines the contributions of several features into **one numerical prediction**.

FCIS — For Course Use Only

Interpreting Coefficients

FCIS — For Course Use Only

What does a weight mean?

In multiple linear regression, a coefficient w_j represents the **change in predicted target** for a one-unit increase in x_j, **holding other features fixed**.

Important interpretation cautions

Issue	Why it matters
Different feature scales	Coefficient magnitudes are **not directly comparable**
Correlated features	Individual effects may be **difficult to separate**
Nonlinear relationships	A constant slope may **oversimplify the pattern**
Feature interactions	One feature's effect may **depend on another**
Categorical encoding	Coefficients depend on the **reference category**

FCIS — For Course Use Only

💡 **Key idea.** Coefficients describe **model associations**, not necessarily causal effects. Interpretation requires attention to **units, other features, and domain context**.

Model Assumptions

FCIS — Private Use Only

When does linear regression work well?

Linear regression is most appropriate when its **linear representation captures the main relationship** in the data.

Model and Data

- **Approximate linearity:** The relationship is reasonably represented by the features.
- **Relevant inputs:** Features are measured and encoded appropriately.
- **Influential observations:** Extreme values do not unduly distort the fit.

Error Behavior

- **No systematic bias:** Errors should not follow a predictable pattern given the features.
- **Stable variance:** Similar error spread is useful, especially for standard inference.
- **Independence:** Dependence between observations matters for uncertainty estimates.

FCIS — For Course Use Only

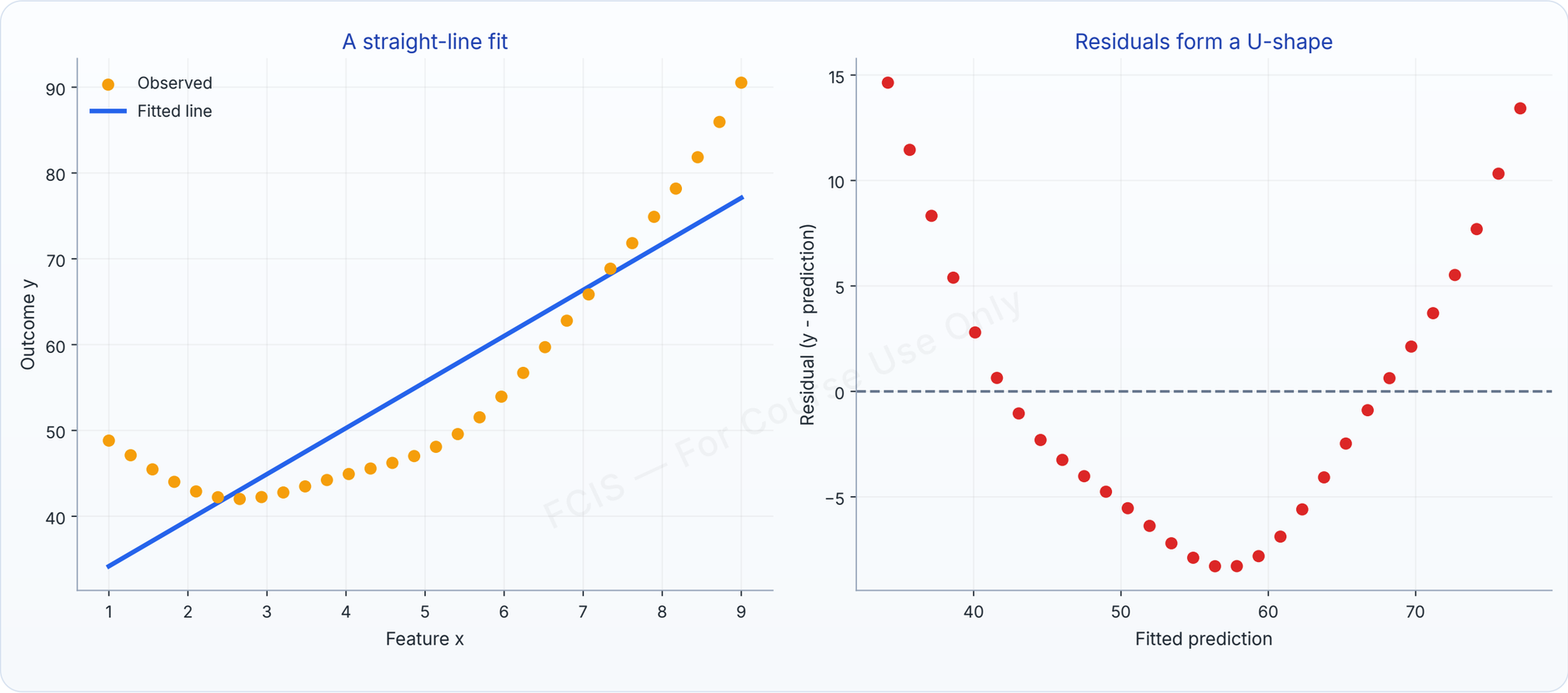
💡 **Key idea.** Assumptions help explain when a linear model may fail. Use **residual diagnostics** to inspect model behavior and **held-out data** to assess predictive generalization.

Residual Diagnostics

FCIS — For Course Use Only

Detecting a missing nonlinear relationship

A **residual plot can reveal structure** that a fitted straight line fails to capture.



💡 **Key observation.** The **U-shaped residual pattern** suggests that the straight-line model has missed a nonlinear relationship. Residuals scattered around zero without a systematic pattern would be more reassuring.

Part III

FCIS — For Private Use Only

Loss Functions

A loss function turns **prediction errors** into a numerical objective that the **learning algorithm can minimize**.

FCIS — For Course Use Only

Why We Need a Loss Function

FCIS — Course Use Only

From prediction errors to a learning objective

Many lines can fit the same data. A **loss function** gives us a numerical way to compare them.

Concept	Meaning	Example
Residual	Signed error: $e_i = y_i - \hat{y}_i$	$e_i = -4$
Loss	Penalty for one prediction: $\ell_i = e_i^2$	$\ell_i = 16$
Objective	Average loss across training examples	$MSE = \frac{1}{n}\sum_i \ell_i$

CIS — Course Use Only

Why distinguish them?

- **Residuals** indicate the direction of prediction errors.
- **Losses** measure how costly individual errors are.
- **Training objectives** summarize errors to guide learning.

Other choices are possible, such as **absolute error** $|e_i|$.

> 💡 **Key idea.** The choice of loss determines **what the model considers a good fit** and how prediction errors influence learning.

Mean Squared Error

FCIS — Course Use Only

A common regression objective

Mean Squared Error (MSE) averages the squared differences between actual and predicted values:

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

Why square the errors?

- **No cancellation:** Positive and negative errors contribute positively.
- **Larger penalties:** Large errors receive greater weight.
- **Easy optimization:** The squared-error objective is smooth and differentiable.

Important limitation

Outlier sensitivity. Because errors are squared, a few unusually large errors can **dominate the loss** and strongly influence the fitted model.

Worked Example: MSE

FCIS — Course Use Only

Calculate the average squared error

Suppose a regression model makes three predictions:

Observation	Actual y	Predicted $\hat{y}$	Residual e	e^2
1	70	68	$+2$	4
2	80	76	$+4$	16
3	90	95	-5	25

Step 1: Mean Squared Error

$$MSE = \frac{4 + 16 + 25}{3} = \boxed{15}$$

FCIS — For Course Use Only

Step 2: Root Mean Squared Error

$$RMSE = \sqrt{MSE} = \sqrt{15} \approx \boxed{3.87}$$

💡 **Key idea.** MSE is measured in **squared target units**, while **RMSE uses the original target units**, making the typical error magnitude easier to interpret.

Loss Surface

FCIS — For Reference Use Only

How model parameters affect loss

For simple linear regression:

$$\hat{y} = w_0 + w_1 x$$

The training loss depends on the **intercept** w_0 and **slope** w_1:

$$J(w_0, w_1) = \frac{1}{n} \sum_{i=1}^{n} [y_i - (w_0 + w_1 x_i)]^2$$

For Course Use Only

What does this mean?

- Different **parameter values** produce different prediction lines.
- Each line has a corresponding **MSE value**.
- Training searches for parameters that **minimize the loss**.

💡 **Key idea.** The loss surface shows how error changes with model parameters. **Optimization** finds parameter values that produce a better fit to the training data.

Part IV

FCIS — Personal Use Only

Gradient Descent

Gradient descent learns model parameters by repeatedly taking steps in the direction that **reduces the loss**.

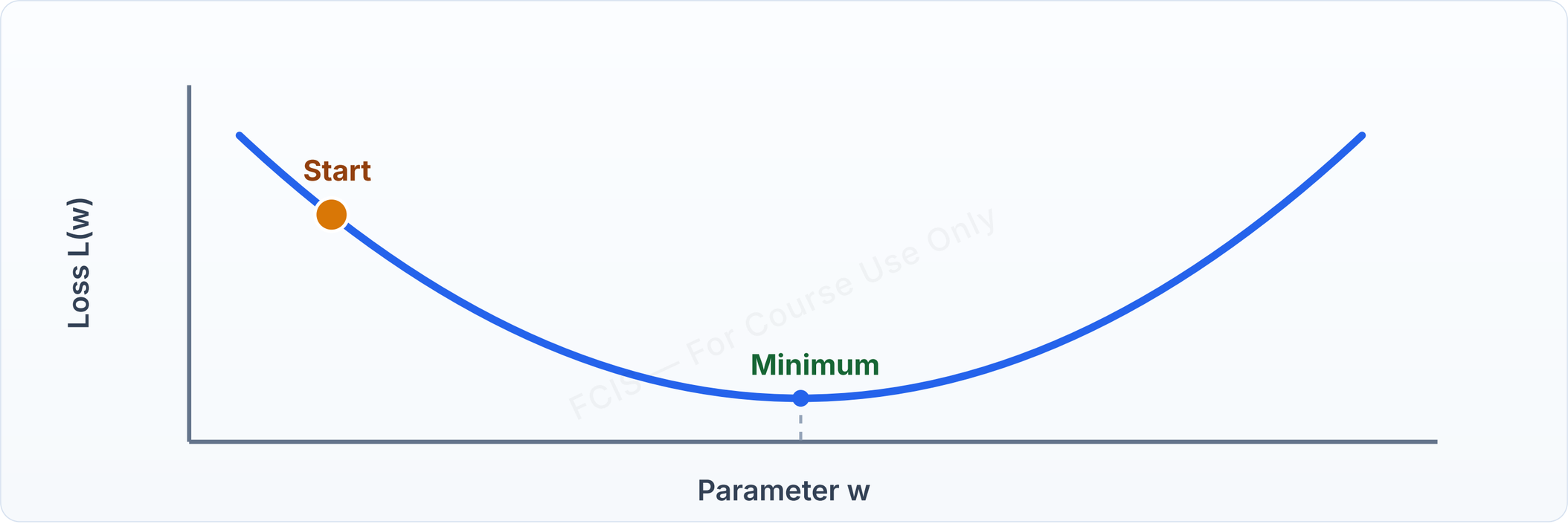
FCIS — For Course Use Only

Gradient Descent Intuition

FCIS — Course Use Only

Follow the slope downhill

The derivative $\frac{dL}{dw}$ tells us how the loss changes with w. The update $w \leftarrow w - \alpha \frac{dL}{dw}$ moves **opposite to the slope**.

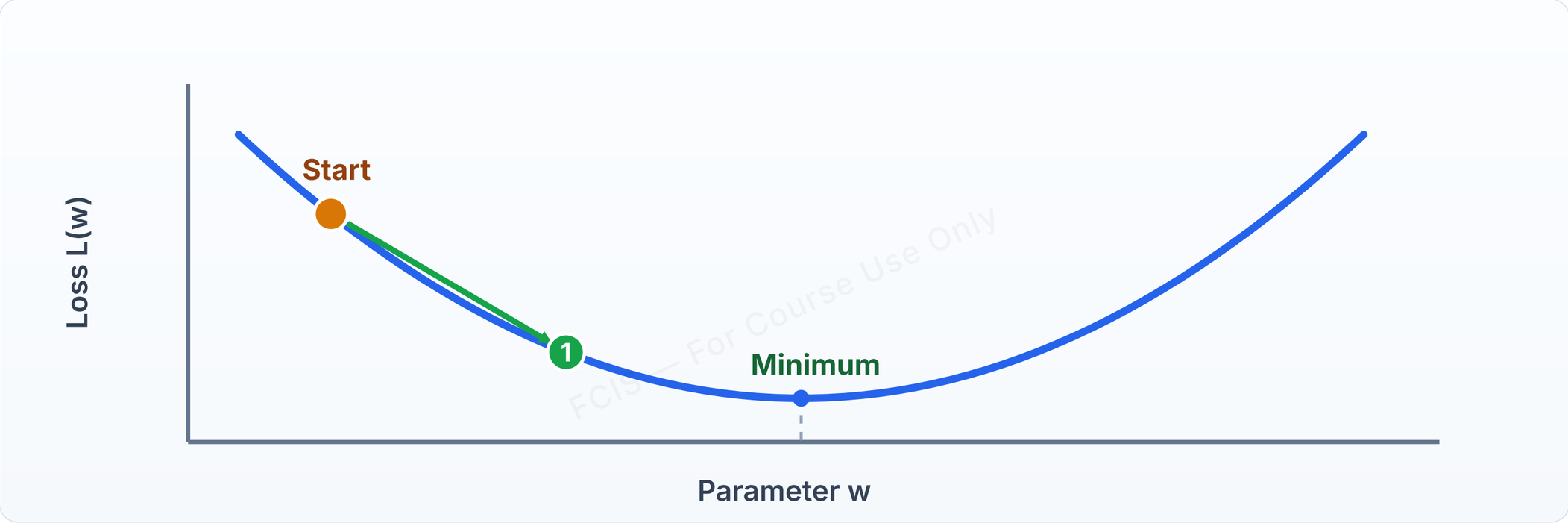


FCIS — For Course Use Only

💡 **Key idea.** A **positive slope** moves w left; a **negative slope** moves w right. As the model approaches the minimum, the gradient becomes smaller.

Follow the slope downhill

The derivative $\frac{dL}{dw}$ tells us how the loss changes with w. The update $w \leftarrow w - \alpha \frac{dL}{dw}$ moves **opposite to the slope**.

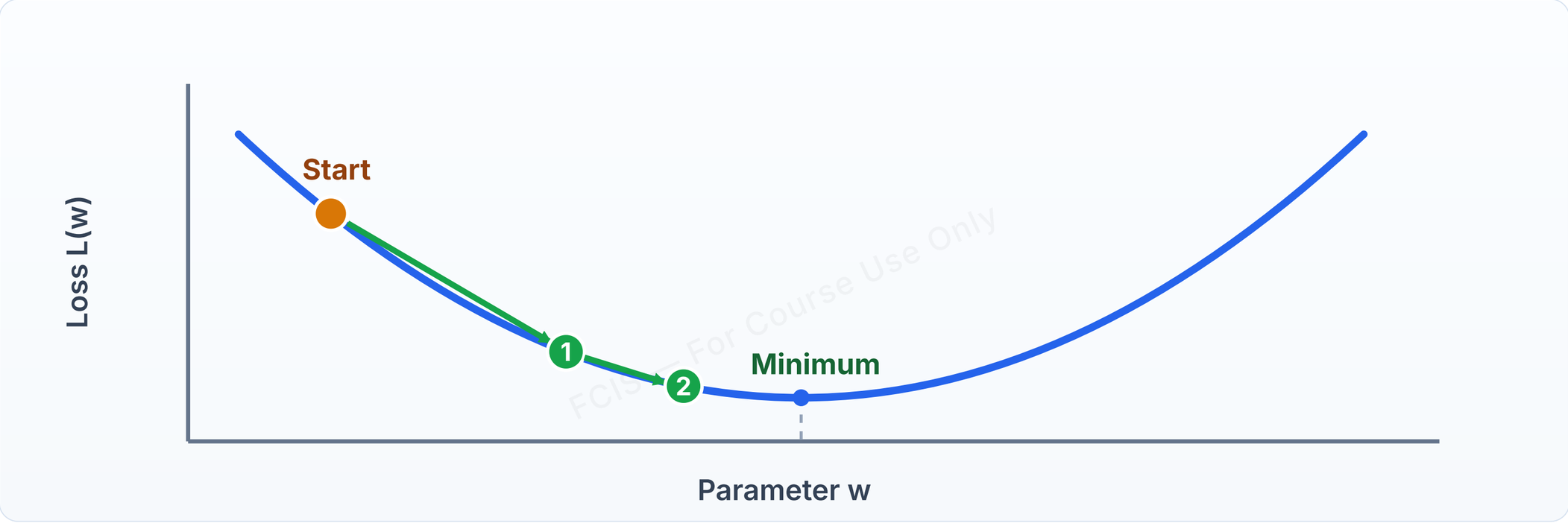


FCIS — For Course Use Only

💡 **Key idea.** A **positive slope** moves w left; a **negative slope** moves w right. As the model approaches the minimum, the gradient becomes smaller.

Follow the slope downhill

The derivative $\frac{dL}{dw}$ tells us how the loss changes with w. The update $w \leftarrow w - \alpha \frac{dL}{dw}$ moves **opposite to the slope**.



FCIS — For Course Use Only

💡 **Key idea.** A **positive slope** moves w left; a **negative slope** moves w right. As the model approaches the minimum, the gradient becomes smaller.

Follow the slope downhill

The derivative $\frac{dL}{dw}$ tells us how the loss changes with w. The update $w \leftarrow w - \alpha \frac{dL}{dw}$ moves **opposite to the slope**.



FCIS — For Course Use Only

💡 **Key idea.** A **positive slope** moves w left; a **negative slope** moves w right. As the model approaches the minimum, the gradient becomes smaller.

Learning Rate

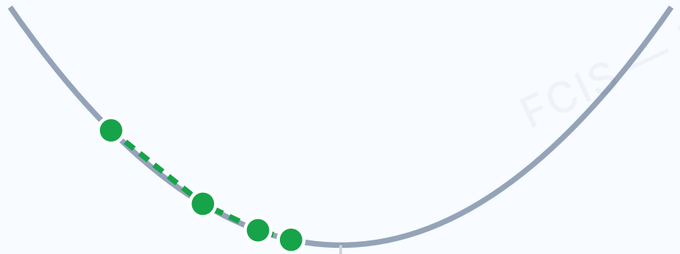
FCIS — For Course Use Only

Choosing the step size

The **learning rate** α determines how far the parameters move during each update in $w \leftarrow w - \alpha \nabla_w L$.

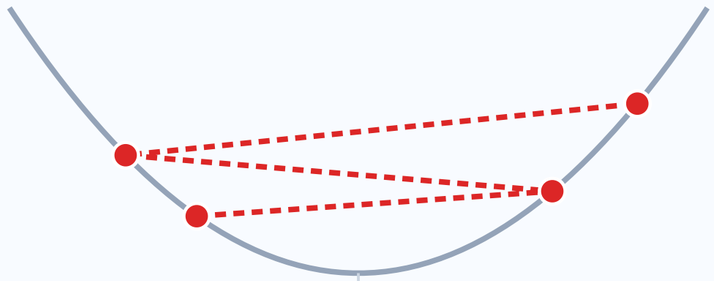
Learning rate	Typical behavior
Too small	Slow progress
Appropriate	Moves toward a minimum
Too large	May overshoot, oscillate, or diverge

Appropriate learning rate



Steps shrink near the minimum

Learning rate too large



Steps overshoot and move farther away

FCIS — For Course Use Only

Important. A learning rate that works for one problem may be too large for another. The appropriate step size depends on the **loss curvature, feature scaling, and optimization method**.

Gradient for Simple Linear Regression

For Course Use Only

One data point

1. Prediction and error

For one training example:

$$\hat{y} = wx + b, \qquad r = \hat{y} - y, \qquad L = r^2$$

2. Compute the gradients

Using the **chain rule**:

$$\frac{\partial L}{\partial w} = 2rx, \qquad \frac{\partial L}{\partial b} = 2r$$

FCIS — For Course Use Only

3. Update the parameters

With learning rate α:

$$w \leftarrow w - \alpha(2rx)$$

$$b \leftarrow b - \alpha(2r)$$

💡 **Key idea.** The residual r determines the direction of correction, while the input x scales the weight gradient. The **learning rate** controls the update size.

Worked Gradient Update

FCIS — For Use Only

One example, one step — and check the loss

For $x = 3$, $y = 10$, and current parameters $w = 2$, $b = 1$:

$$\hat{y} = wx + b = 7, \qquad r = \hat{y} - y = -3, \qquad L = r^2 = 9$$

The gradients of this **one-example squared loss** are:

$$\frac{\partial L}{\partial w} = 2rx = -18 \qquad \frac{\partial L}{\partial b} = 2r = -6$$

Choose $\alpha = 0.02$ and update **both** parameters:

$$w_{\text{new}} = 2 - 0.02(-18) = 2.36 \qquad b_{\text{new}} = 1 - 0.02(-6) = 1.12$$

Check: $\hat{y}_{\text{new}} = 2.36(3) + 1.12 = 8.2$, so $L_{\text{new}} = (8.2 - 10)^2 = 3.24$.

Update	Prediction	Squared loss
Before	7.0	**9.00**
After ($\alpha = 0.02$)	8.2	**3.24**

A larger step, $\alpha = 0.1$, would predict **13**, leaving the squared loss at **9**. An update does not automatically improve the objective.

ECIS — For Course Use Only

Batch Gradient Descent

FCIS — Private Use Only

Use all training examples before updating

Batch **Gradient Descent** computes gradients using the entire training set:

$$\nabla L = \frac{1}{n} \sum_{i=1}^{n} \nabla L_i$$

Advantages

- stable direction;
- directly follows the full **loss surface**;
- good for small datasets.

Limitations

- expensive for very large datasets;
- one update requires scanning all examples;
- can be slow when n is large.

FCIS — For Course Use Only

> 💡 Batch gradient descent is conceptually clean but not always computationally efficient.

Stochastic Gradient Descent

FCIS — Personal Use Only

Update using one example at a time

Stochastic **Gradient Descent** updates parameters using one training example at a time.

Advantages

- fast updates;
- can handle large datasets;
- noisy updates may help escape shallow regions.

Limitations

- update direction is noisy;
- loss curve may fluctuate;
- **learning rate** scheduling becomes important.

ECIS — For Course Use Only

$$w \leftarrow w - \alpha \nabla L_i(w)$$

💡 SGD trades stability for speed and scalability.

Mini-Batch Gradient Descent

FCIS — Course Use Only

A practical compromise

Mini-batch **Gradient Descent** uses a small subset of examples at each update.

If a mini-batch B has m examples:

$$\nabla L_B = \frac{1}{m} \sum_{i \in B} \nabla L_i$$

Why common in practice?

- more stable than SGD;
- less expensive than full batch;
- efficient on modern hardware;
- works naturally with neural networks.

FCIS — For Course Use Only

💡 Mini-batch gradient descent is often the default training strategy for large-scale models.

Comparing Gradient Descent Variants

For Course Use Only

Stability vs. efficiency

The main difference between gradient descent methods is **how many training examples are used to compute each parameter update**.

Method	Examples per update	Advantage	Limitation
Batch GD	Entire training set	Stable, exact training-set gradient	Computationally expensive per update
SGD	One example	Cheap, frequent updates	Noisy, fluctuating updates
Mini-batch GD	Small group of examples	Efficient, less noisy than SGD	Batch size requires tuning

The trade-off

- **Batch GD:** More stable gradients, but expensive updates.
- **SGD:** Cheaper updates, but greater variability.
- **Mini-batch GD:** Balances computational efficiency and gradient stability.

💡 **Key idea.** Mini-batch gradient descent is commonly used to train neural networks because it balances **efficient computation** with **reasonably stable gradient estimates**.

When Optimization Fails

FCIS — Private Use Only

Common causes

What can go wrong?

Optimization difficulties:

- **Learning rate too large:** Unstable updates.
- **Learning rate too small:** Very slow progress.
- **Different feature scales:** Difficult optimization.
- **Ill-conditioned loss surface:** Slow or uneven convergence.
- **Vanishing or exploding gradients:** Weak or unstable learning.

Generalization risk:

- Model too flexible for the available data.

Practical responses

Improve optimization:

- Scale numerical features.
- Tune the learning rate.
- Use mini-batches for efficient updates.
- Monitor training loss.

Improve generalization:

- Add appropriate regularization.
- Monitor validation loss and use **early stopping** when appropriate.

FCIS — For Course Use Only

💡 **Key distinction.** Poor training progress suggests an **optimization problem**; a growing gap between training and validation performance suggests **overfitting**. The appropriate response depends on the problem.

Part V

FCIS — For Private Use Only

Model Flexibility and Overfitting

A model that **fits training data extremely well** may still **generalize poorly**.

FCIS — For Course Use Only

Underfitting and Overfitting

FCIS Course Use Only

Revisit generalization through regression curves

Lecture 1 introduced **underfitting** and **overfitting**. Polynomial regression illustrates how **model flexibility** affects both.

Situation	Training error	Validation error	Regression interpretation
Underfitting	High	High	Curve is too simple to capture the underlying pattern
Good fit	Low	Low	Curve captures the stable trend without following noise
Overfitting	Very low	High	Highly flexible curve fits noise in the training data

FCIS - For Course Use Only

💡 **Key idea.** Increasing polynomial degree can keep reducing **training error**, but may worsen **validation error**. Select model complexity using validation performance, not training error alone.

Polynomial Features

FCIS — Private Use Only

Increasing model flexibility

A linear model in x may be too simple for curved data.

We can add **polynomial features**:

$$x, \quad x^2, \quad x^3, \ldots$$

Then:

$$\hat{y} = w_0 + w_1 x + w_2 x^2 + w_3 x^3$$

The model is still linear in the parameters w_j, but it can represent nonlinear curves in the original input.

For Course Use Only

Risk

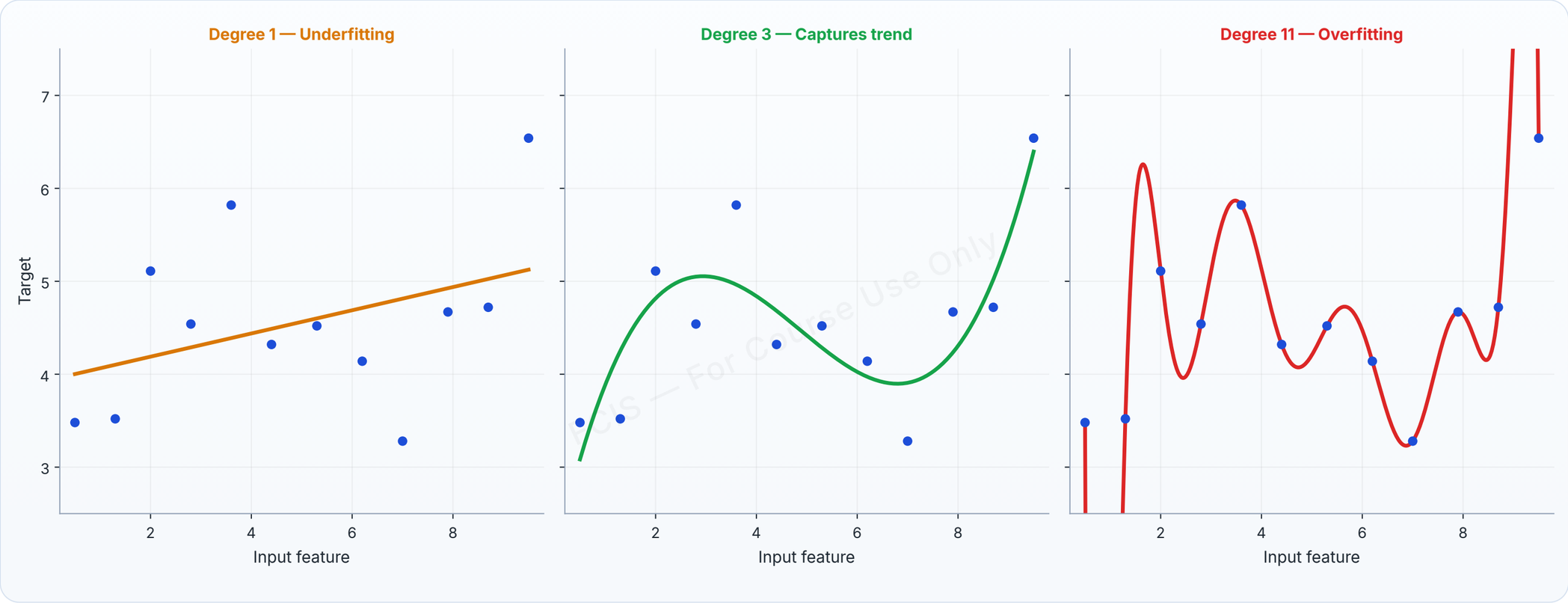
> More polynomial terms can reduce **training error** but increase overfitting.

Flexibility Example

FCIS — Private Use Only

Same observations, three regression curves

All three models are fitted to the **same noisy training observations** (blue dots). Only the **polynomial degree** changes.



Key idea. Degree 1 **misses the nonlinear trend**; degree 3 captures its broad shape; degree 11 closely follows individual observations and may **overfit noise**. Select model complexity using **validation RMSE**, not training fit alone.

Bias–Variance Intuition

FCIS — Course Use Only

A qualitative view

Prediction error can be understood through **three sources**:

Source	Meaning	Typical consequence
High bias	Model is too simple to capture the underlying pattern	**Underfitting**
High variance	Model is too sensitive to the particular training examples	**Overfitting**
Irreducible noise	Randomness or uncertainty that the available features cannot explain	Error that remains even with a good model

FCIS — For Course Use Only

The bias–variance trade-off

- **Simple models:** Often have higher bias and lower variance.
- **Flexible models:** Often have lower bias and higher variance.
- **Goal:** Find the complexity that generalizes best to unseen data.

💡 **Key idea.** Increasing model flexibility often **reduces bias but increases variance**. Validation performance helps us find a useful balance.

Part VI

FCIS — Educational Use Only

Regularization

Regularization discourages **unnecessarily complex models** by adding a **penalty to the loss**.

FCIS — For Course Use Only

Why Regularize?

FCIS — Private Use Only

Control model complexity

Ordinary regression minimizes **training loss**. Regularization adds a penalty to discourage overly complex models.

Regularized objective

$$L_{\text{total}} = \underbrace{L_{\text{data}}}_{\text{Training loss}} + \lambda \underbrace{\Omega(w)}_{\text{Complexity penalty}}$$

Role of λ

- $\lambda = 0$: No regularization penalty.
- **Small λ:** Emphasizes fitting the training data.
- **Large λ:** Penalizes complexity more strongly, potentially causing underfitting.
- **Selection:** Choose λ using **validation performance**.

Here, $\Omega(w)$ measures model complexity, such as the sum of squared weights in **L2 regularization**.

> 💡 **Key idea.** Regularization trades some training fit for lower model complexity, aiming to improve **generalization**. Stronger regularization is not always better.

AI&C — For Course Use Only

Ridge Regression

FCIS — Private Use Only

Penalize large squared coefficients

Ridge regression adds an **L2 penalty** to the training loss:

$$L_{\text{ridge}} = MSE + \lambda \sum_{j=1}^{p} w_j^2$$

Effect

- shrinks coefficients toward zero;
- generally retains all features;
- reduces sensitivity to noise and correlated predictors;
- can improve **generalization**.

When useful. Ridge is often a good choice when **many features contribute small amounts of predictive information**.

For Course Use Only

Lasso Regression

FCIS — For Private Use Only

Penalize absolute coefficient size

Lasso regression adds an **L1 penalty** to the training loss:

$$L_{\text{lasso}} = MSE + \lambda \sum_{j=1}^{p} |w_j|$$

Effect

- shrinks coefficients toward zero;
- can set some coefficients **exactly to zero**;
- performs automatic **feature selection**;
- can be useful when relatively few features are important.

Slides — For Course Use Only

💡 **Key difference.** Unlike Ridge, **Lasso can produce a sparse model** by assigning zero coefficients to some features.

Ridge vs. Lasso

FCIS — For Course Use Only

Similar goal, different behavior

Both methods control complexity by penalizing regression coefficients.

Aspect	Ridge (L2)	Lasso (L1)
Penalty	$\sum_j w_j^2$	$\sum_j \lvert w_j \rvert$
Coefficients	Shrinks toward zero	Some may become exactly zero
Feature selection	Generally no	Yes, often
Useful when	Many features contribute	A smaller subset may matter
Correlated features	Often distributes weights	May select one over others

For Classroom Use Only

💡 **Key idea.** Ridge shrinks coefficients; Lasso can also select features. **Standardize numerical inputs** before fitting either model, and select λ using validation performance.

Regularization Strength

FCIS — Course Use Only

What happens as λ changes?

The parameter $\lambda \geq 0$ controls how strongly coefficient size is penalized.

λ	Expected behavior
0	No regularization; ordinary least squares
Small	Mild coefficient shrinkage
Moderate	May improve generalization
Very large	Strong shrinkage; increased risk of underfitting

FCIS — For Course Use Only

Selection principle

Choose λ using **validation performance**, not training error alone.

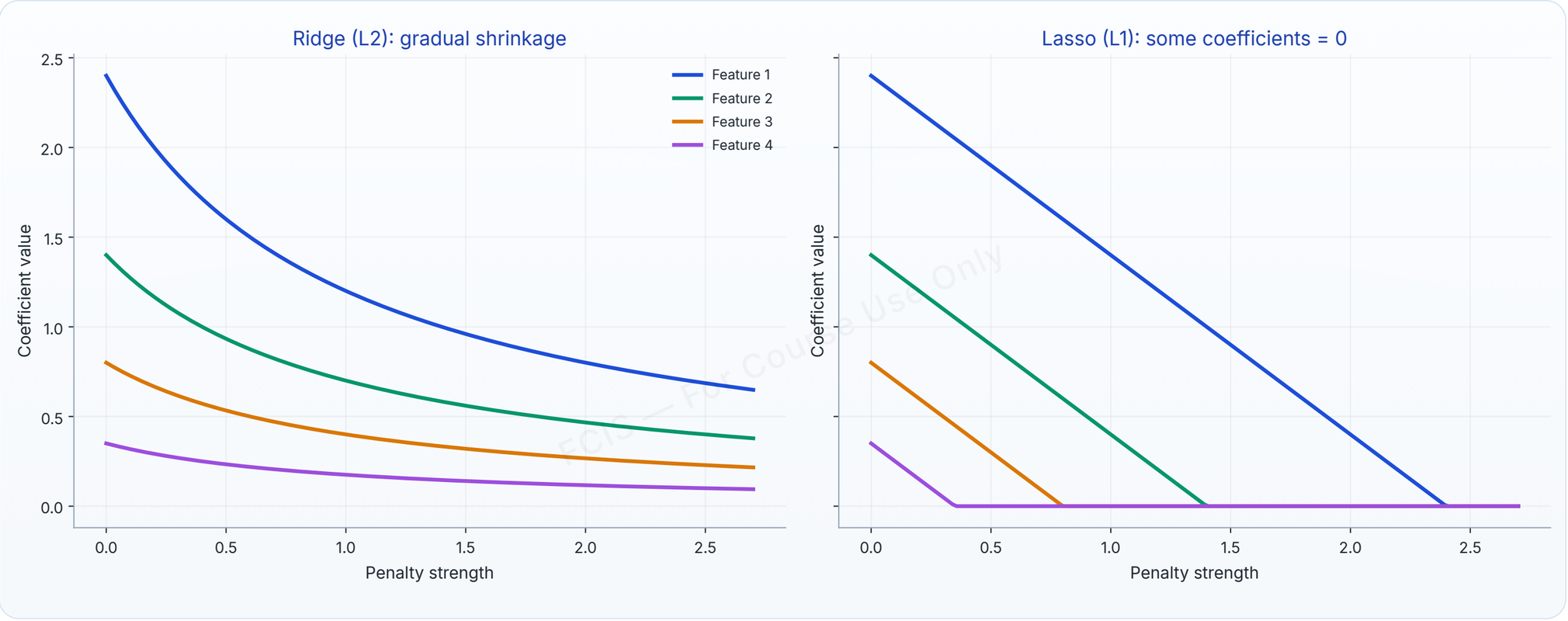
Important. Stronger regularization is not always better. Excessive shrinkage can increase **bias** and cause underfitting.

Regularization Paths

FCIS — Use Only

Ridge shrinks continuously; Lasso can remove coefficients

The curves use **an illustrative orthogonal, standardized-feature case** to isolate how increasing the penalty changes coefficients.



💡 **Key idea. Lasso (L1)** can shrink coefficients **exactly to zero**, producing sparse models. **Ridge (L2)** shrinks coefficients toward zero but generally keeps them **nonzero**. The coefficient paths depend on the **data, feature scaling, and penalty strength**.

Part VII

FCIS — Students Use Only

Regression Evaluation

Evaluate regression models by measuring **prediction errors**, comparing performance on **unseen data**, and choosing metrics that reflect the **application's goals**.

FCIS — For Course Use Only

MAE

FCIS — Personal Use Only

Mean Absolute Error

Mean Absolute Error (MAE) measures the average absolute difference between actual and predicted values:

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

Interpretation and advantages

- **Easy to understand:** Measures the average size of prediction errors.
- **Original units:** Expressed in the **same units as the target**.
- **Equal penalty per unit:** An error of 10 contributes twice as much as an error of 5.
- **No cancellation:** Positive and negative errors both contribute positively.

FCIS - For Course Use Only

Limitation

- The absolute-error function is **not differentiable at zero**, which can affect some optimization methods.

💡 **Key idea.** MAE tells us **how far predictions are from actual values, on average**, regardless of whether the model overpredicts or underpredicts.

MSE and RMSE

FCIS — Private Use Only

Penalizing Larger Errors

Mean Squared Error (MSE) and **Root Mean Squared Error (RMSE)** measure prediction errors:

$$\mathrm{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

$$\mathrm{RMSE} = \sqrt{\mathrm{MSE}}$$

Interpretation

- **MSE** gives **greater weight to large errors** because errors are squared.
- **RMSE** expresses prediction error in the **original target units**.
- Compared with **MAE**, RMSE is **more sensitive to large errors and outliers**.

FCIS - For Course Use Only

💡 **Key idea.** Prefer **RMSE** when large prediction errors deserve extra emphasis. Use **MAE** when a measure less sensitive to extreme errors is more appropriate.

Coefficient of Determination

FCIS — Course Use Only

Understanding R^2

The **coefficient of determination** (R^2) compares a model's squared prediction errors with those of a baseline that always predicts the **mean target value**.

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}$$

How to interpret R^2

Value	Interpretation
$R^2 = 1$	Perfect predictions on the evaluated data
$R^2 = 0$	Same squared error as predicting the mean
$R^2 < 0$	Worse than predicting the mean
$0 < R^2 < 1$	Better than the mean baseline

Important. A high **training** R^2 does not guarantee good predictions on unseen data. Use **validation data** to select the model and a separate **test set** to assess its final performance.

FCIS — For Course Use Only

Choosing a Regression Metric

FCIS — Private Use Only

The metric should match the application

Situation	Possible metric emphasis
Errors have linear cost	MAE
Large errors are especially bad	RMSE
Need variance-explained summary	R^2
Need easy unit interpretation	MAE or RMSE
Comparing across different targets	normalized or relative metrics

FCIS — For Course Use Only

Example

In medical dosage prediction, a few very large errors may be dangerous. **RMSE** or application-specific safety metrics may be important.

In rent estimation, MAE may be easy to explain:

> “The model is off by about 350 SAR on average.”

Evaluation Workflow

FCIS — Internal Use Only

Train, validate, test

A reliable evaluation workflow separates **model learning**, **model selection**, and **final assessment**.

1. Training

Fit preprocessing on training data. Train the model using training examples.

2. Validation

Tune hyperparameters. Compare models using validation performance.

3. Testing

Evaluate the selected model once. Report metrics and interpret them in context.

For Course Use Only

💡 **Key idea.** Use **training data** to learn parameters, **validation data** to select models, and the untouched **test set** for final evaluation. Never use test results to guide model selection.

Part VIII

FCIS — Lecture Use Only

Worked Mini-Case

Apply the complete **regression pipeline** to a practical example: predicting **monthly apartment rent**.

FCIS — For Course Use Only

Mini-Case: Apartment Rent

FCIS — Course Use Only

Problem formulation and preprocessing

A real-estate website wants to predict **monthly apartment rent** from property characteristics.

Input features

Feature	Type	Possible preprocessing
Area	Numerical	Scaling
Number of rooms	Discrete numerical	Keep as count or scale
District	Nominal categorical	One-hot encoding
Building age	Numerical	Scaling; possible nonlinear terms
Furnished	Binary	Encode as 0/1

FCIS — For Course Use Only

Target: $y = \text{monthly rent}$

Task: Supervised regression. The model learns from apartments with **known rents** and predicts rent for **new apartments**.

Fit preprocessing steps using **training data only**. Scaling is particularly important when applying **Ridge or Lasso regularization**.

Mini-Case: Candidate Models

FCIS — Course Use Only

Compare complexity and generalization

Model	Main characteristic
Linear regression	Additive linear effects
Polynomial features + regression	Captures selected nonlinear patterns
Ridge regression (L2)	Shrinks coefficients to control complexity
Lasso regression (L1)	Can shrink some coefficients exactly to zero

Evaluation plan

1. **Train** models using the training set.
2. **Validate** feature transformations, polynomial degree, and regularization strength λ.
3. **Select** the model based on validation performance.
4. **Test** the selected model on held-out data.
5. **Report MAE and RMSE** in rent units and assess practical usefulness.

For Course Use Only

💡 **Key idea.** Prefer the model that **generalizes well**, not necessarily the one with the lowest training error.

Mini-Case: Worked Validation Comparison

For Course Use Only

Decide using evidence, not training fit

Suppose the apartment-rent pipeline is fitted on training data, with model choices and hyperparameters selected by **validation**. These are **hypothetical classroom results**, not measurements from a real dataset.

Candidate model	Validation MAE (SAR/month)	Validation RMSE (SAR/month)	Interpretation
Linear regression	540	720	Simple baseline
Polynomial features + linear regression	465	640	Captures useful curvature
Ridge regression	**430**	580	Lowest average absolute error
Lasso regression	450	**545**	Fewer large errors by RMSE

For Course Use Only

Example reasoning

- If typical absolute rent error matters most, **Ridge** is the first candidate to investigate.
- If larger errors are more costly, **Lasso** has the lower RMSE in this illustration.
- Do **not** use the test set to settle this decision: choose the metric in advance, select on validation, and test the chosen workflow once.

💡 A numerical comparison is evidence, not an automatic winner. Consider the metric, uncertainty in the estimates, interpretability, and deployment requirements.

Mini-Case: Reasoning Questions

FCIS — Course Use Only

Explain and Justify Your Choices

Features and Modeling

1. Why must **district** be encoded rather than treated as a numerical quantity?
2. Why might **area** and **building age** need scaling?
3. Why might building age have a **nonlinear effect** on rent?
4. How can **Ridge regression** help when many district indicators are included?

Evaluation and Selection

5. Why is **MAE** often easier to explain to end users than **MSE**?
6. Using the **illustrative validation table**, which model has the lowest **MAE**? Which has the lowest **RMSE**?
7. Why must the **test set** not be used to select λ or the polynomial degree?

CIS — For Course Use Only

💡 **Key lesson.** A good regression report should **justify preprocessing and model selection**, present meaningful **evaluation metrics**, and explain the practical implications of prediction errors.

Practice & Consolidation

FCIS — For Course Use Only

Regression Modeling and Common Mistakes

Apply the full learning process: calculate predictions, compare models, diagnose **overfitting**, and justify **leakage-safe evaluation**.

FCIS — For Course Use Only

Activity: Compute Predictions

FCIS — For Course Use Only

Manual practice

Suppose:

$$\widehat{\text{score}} = 30 + 3(\text{hours}) + 0.2(\text{attendance}) + 5(\text{GPA})$$

For a student with:

$$\text{hours} = 8 \qquad \text{attendance} = 90 \qquad \text{GPA} = 3.0$$

compute the predicted score.

FCIS — For Course Use Only

Questions

1. What is $\hat{y}$?
2. If the actual score is 82, what is the residual?
3. Did the model overpredict or underpredict?

Manual practice

Suppose:

$$\widehat{\text{score}} = 30 + 3(\text{hours}) + 0.2(\text{attendance}) + 5(\text{GPA})$$

For a student with:

$$\text{hours} = 8 \qquad \text{attendance} = 90 \qquad \text{GPA} = 3.0$$

compute the predicted score.

FCIS — For Course Use Only

Questions

1. What is $\hat{y}$?
2. If the actual score is 82, what is the residual?
3. Did the model overpredict or underpredict?

> $\hat{y} = 30 + 3(8) + 0.2(90) + 5(3) = 87$. Residual $= 82 - 87 = -5$, so the model overpredicted by 5.

Activity: Choose the Better Model

FCIS — Internal Use Only

Compare validation metrics

Two rent-prediction models were evaluated on validation data.

Model	MAE	RMSE	Interpretation
A	410	690	
B	460	540	

Questions

1. Which model has smaller average **absolute error**?
2. Which model has fewer very large errors?
3. Which one would you choose if large errors are very costly?
4. Why is there not always a single obvious answer?

FCIS — For Course Use Only

Compare validation metrics

Two rent-prediction models were evaluated on validation data.

Model	MAE	RMSE	Interpretation
A	410	690	
B	460	540	

Questions

1. Which model has smaller average **absolute error**?
2. Which model has fewer very large errors?
3. Which one would you choose if large errors are very costly?
4. Why is there not always a single obvious answer?

FCIS — For Course Use Only

> Model A has lower MAE, while Model B has lower **RMSE**. If large errors are especially costly, Model B may be preferable.

Activity: Detect Overfitting

FCIS — Private Use Only

Training vs. validation performance

Model	Training RMSE	Validation RMSE
Linear	18.5	20.1
Degree 3 polynomial	10.2	14.0
Degree 12 polynomial	1.1	38.7

Questions

1. Which model underfits?
2. Which model likely overfits?
3. Which model appears best by validation RMSE?
4. Why should we not choose the model with the lowest training error?

FCIS — For Course Use Only

Training vs. validation performance

Model	Training RMSE	Validation RMSE
Linear	18.5	20.1
Degree 3 polynomial	10.2	14.0
Degree 12 polynomial	1.1	38.7

Questions

1. Which model underfits?
2. Which model likely overfits?
3. Which model appears best by validation RMSE?
4. Why should we not choose the model with the lowest training error?

FCIS — For Course Use Only

Degree 12 likely overfits. Degree 3 appears best by validation RMSE. The lowest **training error** is not the goal.

Common Mistakes

FCIS — Personal Use Only

Pitfalls in regression modeling

Common mistake	Why it matters
Evaluating training error only	Hides **overfitting** and poor generalization
Using target-derived features	Causes **data leakage** and misleading results
Ignoring feature scaling with Ridge or Lasso	Makes regularization affect features **unequally**
Repeatedly selecting models using test error	Compromises the **independence of the test set**
Ignoring coefficient units	Leads to **misleading comparisons**
Removing outliers automatically	May discard **valid and important observations**
Reporting MSE without explaining units	Squared units make errors **harder to interpret**
Assuming linearity without checking residuals	May miss **nonlinear patterns**

FCIS — For Course Use Only

💡 **Key lesson.** Reliable regression requires **leakage-free preprocessing**, appropriate model selection, meaningful evaluation metrics, and careful **residual analysis**.

Mini-Quiz

FCIS — For Course Use Only

Check your understanding

1. What is the difference between a prediction error and a loss?
2. Why does MSE penalize large errors more than MAE?
3. What does the learning rate control in gradient descent?
4. Why can polynomial features cause overfitting?
5. What is the main difference between Ridge and Lasso?
6. Why should λ be chosen using validation data?
7. Which metric is more interpretable in the original target unit: MSE or RMSE?
8. What does $R^2 < 0$ mean?

FCIS — For Course Use Only

Check your understanding

1. What is the difference between a prediction error and a loss?
2. Why does MSE penalize large errors more than MAE?
3. What does the learning rate control in gradient descent?
4. Why can polynomial features cause overfitting?
5. What is the main difference between Ridge and Lasso?
6. Why should λ be chosen using validation data?
7. Which metric is more interpretable in the original target unit: MSE or RMSE?
8. What does $R^2 < 0$ mean?

FCIS — For Course Use Only

Suggested answers: a residual is the signed prediction error, a per-example loss penalizes that error, and an objective (such as MSE) aggregates losses; MSE squares errors; **learning rate** controls update size; high-degree polynomials can fit noise; Ridge shrinks, Lasso can zero coefficients; hyperparameters need validation; RMSE has original units; the model is worse than predicting the mean.

Lecture 3 Summary

FCIS — For Course Use Only

Key ideas to retain

Linear regression and optimization

- Regression predicts a **continuous target**.
- Linear regression uses:

$$\hat{y} = w^\top x + b$$

- Residuals measure prediction errors.
- MSE is a common regression objective.
- Gradient descent updates parameters to reduce loss.
- Batch, stochastic, and mini-batch GD differ in how much data each update uses.

Generalization and evaluation

- More flexibility can reduce training error but increase overfitting.
- Polynomial features can represent nonlinear patterns.
- Ridge and Lasso add penalties to control complexity.
- MAE, MSE, RMSE, and R^2 describe regression performance differently.
- Model choice should be based on validation performance and application context.

FCIS — For Course use Only

End-of-Lecture Check

FCIS — Private Use Only

Five questions to test your understanding

1. How does a **linear regression** model make a prediction?
2. What does gradient descent change during training?
3. Why does scaling matter when regularization is used?
4. How can polynomial features produce both better fits and overfitting?
5. Which regression metric would you report to a non-technical stakeholder, and why?

ECIS - For Course Use Only

💡 **Preparation for next lecture:** Review the sigmoid function, probability interpretation, binary classification, and the difference between regression and classification.

Suggested Reading

FCIS — Private Use Only

Primary and supporting references

Primary reference

G. James, D. Witten, T. Hastie, and R. Tibshirani, *An Introduction to Statistical Learning with Applications in Python*, Springer, 2023.

- Review chapters/sections on linear regression, model flexibility, training/test error, and regularization.

Supporting references

A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, O'Reilly, 2022.

- Review linear models, gradient descent, polynomial regression, and regularized linear models.

K. P. Murphy, *Probabilistic Machine Learning: An Introduction*, MIT Press, 2022.

- Use selected sections for a deeper probabilistic view of regression, loss functions, and optimization.

For Course Use Only

💡 When reading, focus on how prediction, loss, optimization, and generalization form one connected learning process.